Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

Materia: Fundamentación de robótica

Clave: TE3001B

Grupo: 101

Profesores:
Alfredo García Suárez
Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios

Actividad 2.2 PWM con ESP32

Alumnos
Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

2 de Marzo De 2026

Resumen

En el presente trabajo se desarrolló e implementó un sistema de control para un motor DC de 6V utilizando una placa ESP32 integrada con micro-ROS, permitiendo su comunicación dentro del ecosistema ROS 2. El objetivo principal fue regular la velocidad y dirección del motor mediante una señal PWM generada por el microcontrolador y transmitida a través de un puente H L298. Para ello, se diseñó la solución tanto a nivel de hardware como de software, normalizando la señal de control en el intervalo [-1,1] para representar de manera unificada la magnitud y el sentido de giro. Se realizaron pruebas experimentales que permitieron verificar la correcta comunicación entre nodos, así como analizar el comportamiento físico del motor, identificando fenómenos como la zona muerta y la vibración en valores pequeños de entrada. Los resultados obtenidos demuestran el cumplimiento de los objetivos planteados y evidencian la adecuada integración entre el sistema embebido y la arquitectura ROS 2.

Objetivos

Objetivo General

Migrar un sistema de control de velocidad de un motor de corriente directa (DC), originalmente implementado de manera nativa en un ESP32 mediante modulación por ancho de pulso (PWM), hacia una arquitectura distribuida basada en micro-ROS, permitiendo su integración dentro del ecosistema de ROS 2, e implementando comunicación mediante publishers y subscribers, una máquina de estados con reconexión automática y manejo adecuado de memoria dinámica en mensajes tipo String.

Objetivos específicos

- Analizar el funcionamiento del sistema original de control de velocidad por PWM en el ESP32, identificando las variables críticas: duty cycle, valor PWM, voltaje ADC y estado del motor.
- Explicar la resolución del ADC del ESP32 y corregir el error que generaba una señal PWM incorrecta observada en el osciloscopio.
- Implementar un nodo embebido en el ESP32 usando micro-ROS para integrarlo al entorno de ROS 2.

- Configurar publishers para /duty_cycle, /voltaje y /motor_state, y un subscriber para /motor_command.
- Validar el comportamiento del sistema utilizando rqt_plot, comparando la variación del potenciómetro con la respuesta publicada.

Introducción

En los sistemas embebidos aplicados a robótica y automatización, el control de motores de corriente directa (DC) es una de las aplicaciones más comunes, ya que permite regular velocidad y dirección de movimiento en múltiples dispositivos. Tradicionalmente, este control se implementa de forma local en un microcontrolador, donde las señales de entrada son procesadas y convertidas en señales de salida que accionan el actuador. Sin embargo, en aplicaciones robóticas modernas se busca integrar estos sistemas a arquitecturas distribuidas que permitan monitoreo, control remoto y escalabilidad.

Funcionamiento del PWM

Una técnica fundamental para el control de velocidad de motores DC es la Modulación por Ancho de Pulso (PWM, Pulse Width Modulation). El PWM consiste en una señal digital periódica cuya característica principal es el ciclo de trabajo (duty cycle), que representa el porcentaje de tiempo que la señal permanece en nivel alto dentro de un periodo. Al variar el duty cycle, se modifica el valor promedio de voltaje entregado al motor, permitiendo regular su velocidad sin necesidad de variar físicamente la fuente de alimentación. En el caso del ESP32, el PWM se genera mediante canales configurables donde se define la frecuencia y la resolución en bits, lo que determina el rango de valores posibles (por ejemplo, 8 bits equivalen a valores entre 0 y 255).

Funcionamiento del Puente H

Para controlar la dirección de giro del motor se utiliza un puente H, el cual es un arreglo electrónico de transistores que permite invertir la polaridad aplicada al motor. Dependiendo de la combinación de señales de control, el motor puede girar hacia la derecha, hacia la izquierda o permanecer detenido. Este circuito es esencial cuando se requiere control

bidireccional, ya que el microcontrolador por sí solo no puede suministrar la corriente necesaria para el accionamiento directo del motor.

Funcionamiento de Micro-ROS:

Con el objetivo de integrar este sistema a un entorno robótico más amplio, se emplea micro-ROS, una extensión de ROS 2 diseñada para dispositivos con recursos limitados como microcontroladores. Micro-ROS permite que el ESP32 funcione como un nodo dentro de una red ROS 2, publicando y suscribiéndose a tópicos mediante un agente que se ejecuta en una computadora anfitriona. Internamente, utiliza el protocolo DDS-XRCE, optimizado para entornos con restricciones de memoria y procesamiento.

Dentro de esta arquitectura distribuida, el sistema no solo genera la señal PWM para el motor, sino que también publica variables críticas como el duty cycle, el voltaje leído y el estado del motor, además de suscribirse a comandos externos para controlar la dirección y encendido. Para garantizar robustez en la comunicación, se implementa una máquina de estados que permite la reconexión automática en caso de pérdida de enlace con el agente, mejorando la tolerancia a fallos del sistema.

Solución del problema

Para cumplir con los objetivos planteados en el reto, se desarrolló una metodología basada en la implementación de un sistema distribuido utilizando ROS 2 y micro-ROS, en el cual el control del motor de corriente directa se realiza desde una computadora externa mientras que la ejecución de las señales de accionamiento se lleva a cabo en el microcontrolador ESP32.

En el apartado del diseño del circuito nos guiamos por el diagrama proporcionado por Manchester Robotics. En la parte inferior de esta sección se presenta el diagrama de conexión del sistema implementado. Dicho diagrama está basado en el esquema proporcionado originalmente; sin embargo, se realizó una modificación en la fuente de alimentación utilizada. Mientras que en el diagrama de referencia se emplea un battery pack para alimentar el driver del motor, en este caso se utilizó una fuente de alimentación regulada (power supply) disponible en el laboratorio, la cual suministra energía directamente al puente H L298. Asimismo, el sistema incluye una computadora conectada al microcontrolador ESP32 para la comunicación mediante micro-ROS. En cuanto a las conexiones de control, el pin

GPIO 4 del ESP32 se encuentra conectado al pin Enable del puente H, el GPIO 18 al pin IN1 y el GPIO 15 al pin IN2. Finalmente, las salidas OUT1 y OUT2 del puente H se conectan al motor DC con encoder, permitiendo su accionamiento conforme a las señales generadas por el microcontrolador.

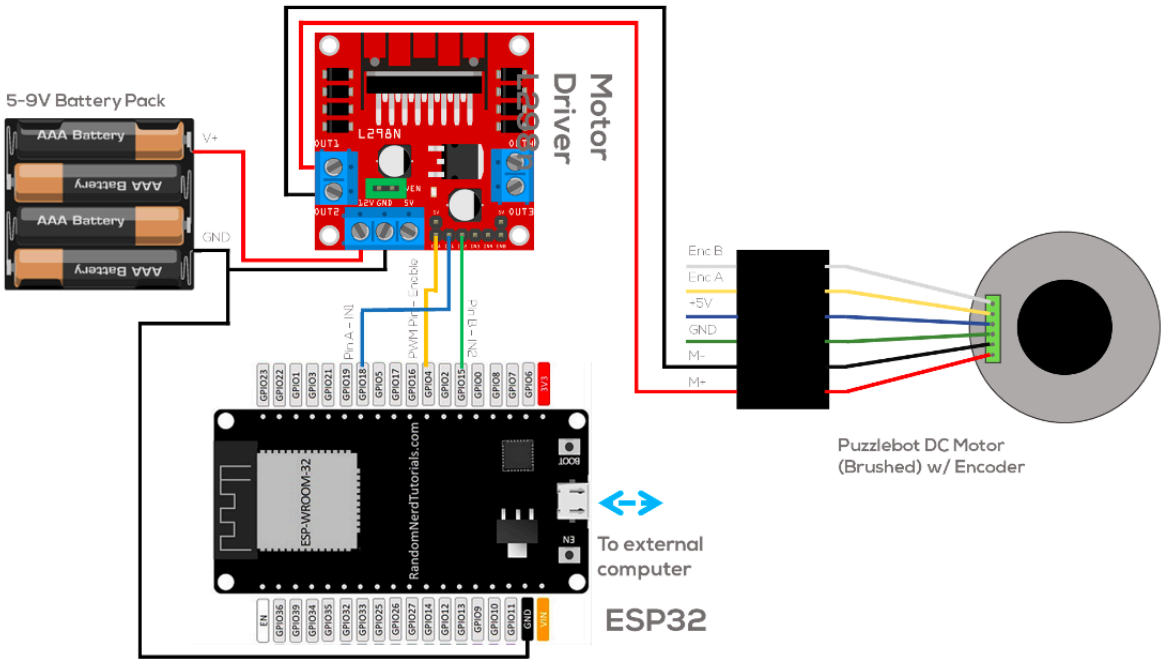

Figura 1. Diagrama de conexiones de la solución implementada.

En cuestión del diseño de la lógica de programación, inicialmente se configuró el entorno de micro-ROS en el ESP32 con el propósito de permitir su integración como un nodo dentro del ecosistema ROS 2. Para ello, se realizó la inicialización del soporte del sistema y la asignación de memoria mediante el allocator por defecto, el cual permite gestionar los recursos necesarios para la creación del nodo, la suscripción y el ejecutor dentro del microcontrolador.

Posteriormente, se implementó el nodo /motor, el cual tiene la función principal de suscribirse al tópico /cmd_pwm, desde donde recibe los comandos necesarios para regular la velocidad y dirección de giro del motor.

Una vez recibidos los datos desde el tópico, el sistema realiza el procesamiento del valor de entrada para determinar dos parámetros fundamentales:

- La dirección de giro del motor.
- El ciclo de trabajo (duty cycle) de la señal PWM.

Para ello, el valor recibido se normaliza dentro del intervalo [-1,1], donde el signo del valor define la dirección de rotación del motor y la magnitud del mismo representa la potencia que será suministrada. Con base en este procesamiento, se generan dos tipos de señales de salida desde el ESP32:

- Una señal digital que define la dirección del motor.
- Una señal PWM encargada de controlar la velocidad del mismo.

La generación de la señal PWM se realizó utilizando los canales LEDC del ESP32, configurando la frecuencia de operación en 980 Hz y una resolución de 8 bits, permitiendo obtener valores de ciclo de trabajo dentro del rango mencionado. Estos valores son posteriormente enviados al driver puente H, el cual suministra la potencia necesaria al motor DC.

Finalmente el flujo de la información desde la computadora hasta el motor DC quedo de la siguiente manera

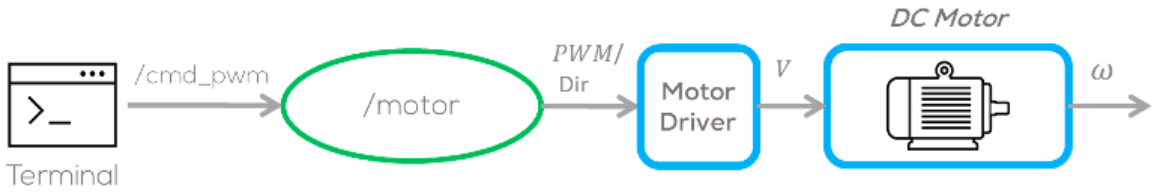

Figura 2. Flujo de información desde ROS 2 hasta el motor DC.

A continuación, se presenta el código completo implementado en el ESP32 para la integración con micro-ROS y el control del motor DC. En este se incluye la configuración del entorno ROS 2, la inicialización del nodo /motor, la suscripción al tópico /cmd_pwm, así como la lógica de procesamiento de la señal recibida para la generación de la señal PWM y el

control de dirección mediante el puente H. El código mostrado corresponde a la versión final utilizada durante las pruebas experimentales descritas anteriormente.

```
#include <Arduino.h>
#include <micro_ros_arduino.h>
#include <rcl/rcl.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>
#include <std_msgs/msg/float32.h>

// Pines
const int pin_in1 = 18;
const int pin_in2 = 15;
const int pin_pwm = 4;

// Configuración PWM ESP32
const int pwmFreq = 980;        // Hz
const int pwmResolution = 8;    // 0-255

// Estructuras micro-ROS
rcl_subscription_t subscriber;
std_msgs_msg_Float32 msg;
rclc_executor_t executor;
rclc_support_t support;
rcl_allocator_t allocator;
rcl_node_t node;

// Macros de verificación de errores
#define RCCHECK(fn) { rcl_ret_t rc = fn; if(rc != RCL_RET_OK){error_loop();
#define RCSOFTCHECK(fn) { rcl_ret_t rc = fn; if(rc != RCL_RET_OK){}}

void error_loop(){
  while(1){ delay(100); }
}

// Callback: recibe referencia de velocidad en rango [-1, 1]
void subscription_callback(const void * msgin) {
  const std_msgs_msg_Float32 * incoming_msg =
    (const std_msgs_msg_Float32 *)msgin;

  float cmd = incoming_msg->data;

  // Saturación de seguridad
  if (cmd > 1.0) cmd = 1.0;
  if (cmd < -1.0) cmd = -1.0;

  // Control de dirección
  if (cmd > 0) {
    digitalWrite(pin_in1, HIGH);
    digitalWrite(pin_in2, LOW);
  }
  else if (cmd < 0) {
    digitalWrite(pin_in1, LOW);
    digitalWrite(pin_in2, HIGH);
  }
  else {
    digitalWrite(pin_in1, LOW);
    digitalWrite(pin_in2, LOW);
```

Figura 3. Sección inicial del código del nodo /motor (configuración y estructuras).

```
    digitalWrite(pin_in2, LOW);
  }

  // Conversión a duty cycle (0-255)
  int dutyCycle = (int)(abs(cmd) * 255.0);

  // Aplicar PWM al motor
  ledcWrite(pin_pwm, dutyCycle);
}

void setup() {

  // Configuración de pines de dirección
  pinMode(pin_in1, OUTPUT);
  pinMode(pin_in2, OUTPUT);

  // Inicialización PWM en el pin seleccionado
  ledcAttach(pin_pwm, pwmFreq, pwmResolution);

  // Inicialización transporte micro-ROS
  set_microros_transports();
  delay(2000);

  allocator = rcl_get_default_allocator();

  // Inicialización del entorno micro-ROS
  RCCHECK(rclc_support_init(&support, 0, NULL, &allocator));
  RCCHECK(rclc_node_init_default(&node, "motor", "", &support));

  // Suscripción al tópico de referencia
  RCCHECK(rclc_subscription_init_default(
    &subscriber,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
    "cmd_pwm"));

  // Configuración del ejecutor
  RCCHECK(rclc_executor_init(&executor, &support.context, 1, &allocator));
  RCCHECK(rclc_executor_add_subscription(
    &executor, &subscriber, &msg,
    &subscription_callback, ON_NEW_DATA));
}

void loop() {
  // Procesa eventos de micro-ROS
  RCCHECK(rclc_executor_spin_some(&executor, RCL_MS_TO_NS(10)));
  delay(15);
}
```

Figura 4. Sección del código correspondiente al callback y control del motor.

Adicionalmente, dentro de los requerimientos del reto se solicitaba la implementación de un sistema de reconexión basado en una máquina de estados finitos, proporcionado por Manchester Robotics, con el objetivo de garantizar la robustez del sistema ante posibles fallos de comunicación entre el microcontrolador y el agente micro-ROS. Esta máquina de estados

permitiría detectar la pérdida de conexión y ejecutar automáticamente intentos de reconexión hasta restablecer la comunicación, evitando la intervención manual durante la operación.

Sin embargo, no fue posible integrar completamente este mecanismo en la práctica debido a que varios equipos trabajaban simultáneamente utilizando configuraciones similares dentro del mismo entorno de red. En particular, se presentaron conflictos relacionados con el ROS_DOMAIN_ID, lo que generó interferencias en los tópicos, ya que múltiples nodos compartían los mismos nombres (por ejemplo, /motor y /cmd_pwm). Esto provocó cruces de comunicación y comportamientos inesperados en la red ROS 2, dificultando la validación adecuada de la máquina de estados de reconexión.

A pesar de esta limitación, el sistema logró mantener una conexión estable durante las pruebas realizadas, permitiendo el cumplimiento de los objetivos principales relacionados con el control del motor. Como mejora futura, se recomienda asignar identificadores de dominio distintos para cada equipo o utilizar espacios de nombres (namespaces) diferentes, con el fin de evitar interferencias y permitir la correcta implementación del mecanismo de reconexión basado en máquina de estados.

¿Por qué normalizar entre −1 y 1?

La normalización del valor de control en el intervalo [-1,1] permite representar simultáneamente la dirección y la magnitud de la señal de control en una sola variable. En este esquema, el signo del valor indica el sentido de giro del motor, mientras que el valor absoluto determina la potencia aplicada mediante el ciclo de trabajo de la señal PWM.

Desde el punto de vista de la magnitud, utilizar un rango de 0 a 1 (en valor absoluto) simplifica el tratamiento matemático del sistema, ya que representa una proporción directa de la potencia máxima disponible. Es decir, un valor de 0.5 indica el 50% de la potencia máxima, independientemente de si el sistema utiliza una resolución PWM de 8 bits (0–255), 10 bits o cualquier otra. Posteriormente, este valor normalizado se escala al rango correspondiente del hardware.

¿Por qué separar dirección y PWM?

Separar la dirección de giro y la señal PWM permite un control más preciso y seguro del motor. La señal PWM únicamente regula la cantidad de energía suministrada al motor (velocidad), mientras que la señal de dirección define el sentido de rotación mediante el control lógico del puente H. Esta separación evita ambigüedades en el control, facilita la implementación del sistema en hardware y previene posibles conflictos eléctricos que podrían surgir si ambas funciones fueran controladas mediante una sola señal.

¿Por qué el motor vibra cuando envío valores pequeños?

La vibración del motor al recibir valores pequeños de PWM se debe principalmente a que el voltaje promedio suministrado no es suficiente para vencer la fricción estática del motor. En estos casos, la señal PWM genera impulsos de corriente que intentan mover el rotor, pero al no alcanzar el par necesario para iniciar el movimiento continuo, el motor entra en un estado de oscilación. Este fenómeno provoca pequeños intentos de movimiento seguidos de detenciones rápidas, lo cual se percibe como vibración.

Además, los motores DC presentan una zona muerta (dead zone) en la que valores bajos de PWM no producen movimiento efectivo, generando únicamente calentamiento y vibraciones mecánicas debido a la conmutación interna del motor.

Resultados

En esta sección se presentan las evidencias experimentales obtenidas durante la ejecución del sistema de control del motor DC mediante el uso de ROS 2 y micro-ROS. Las pruebas realizadas tuvieron como objetivo verificar la correcta comunicación entre la computadora y el microcontrolador ESP32, así como la adecuada interpretación de los comandos enviados al tópico /cmd_pwm.

Para validar el funcionamiento del sistema, se empleó el comando:

- ros2 topic pub /cmd_pwm std_msgs/msg/Float32 "{data: X.X}"

donde el valor X.X representa el valor de control normalizado dentro del intervalo [-1,1], el cual determina tanto la dirección como la magnitud de la señal PWM aplicada al motor.

Al ejecutar este comando desde la terminal, el sistema comenzó la publicación de mensajes en el tópico correspondiente, mostrando en pantalla mensajes como:

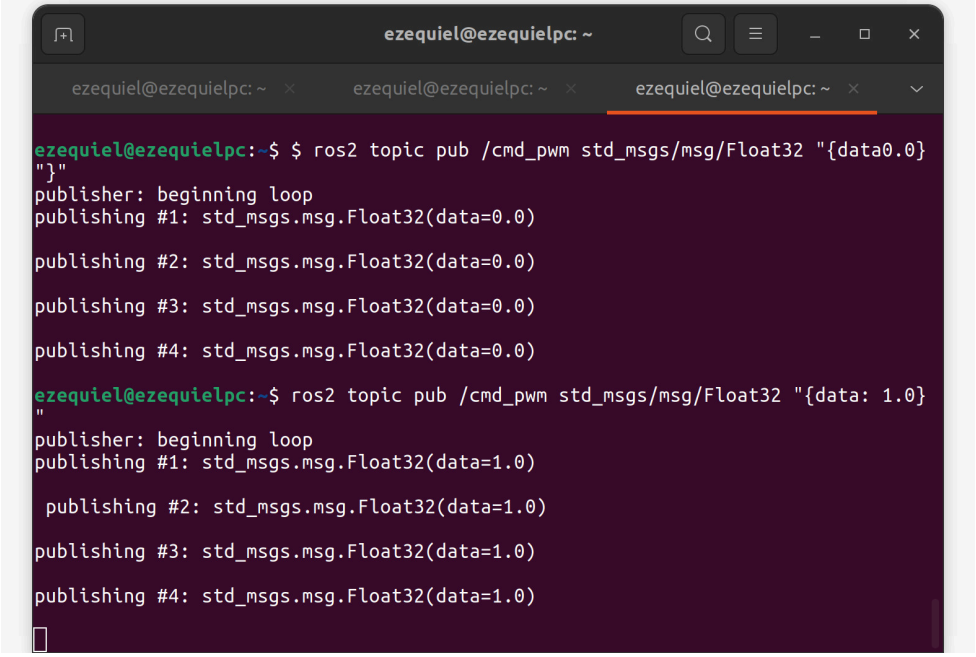

Figura 5. Publicación manual de comandos PWM desde la terminal.

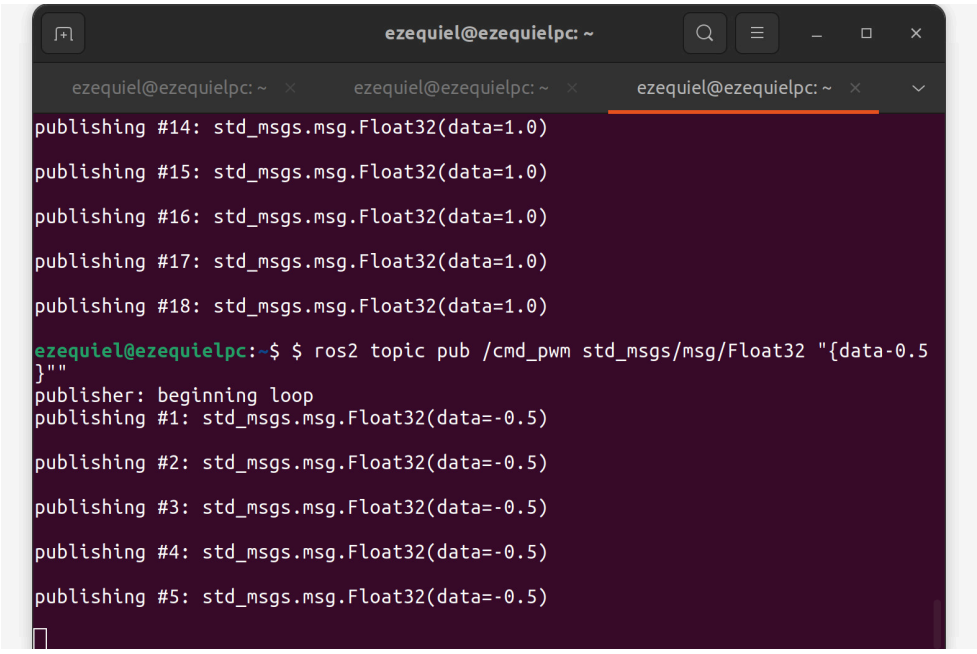

Figura 6. Respuesta del sistema durante la ejecución de pruebas.

Estos mensajes confirman que el nodo publicador está enviando correctamente los datos al sistema, los cuales son posteriormente recibidos por el nodo /motor implementado en el ESP32.

Se realizaron múltiples pruebas variando el valor del dato enviado, tales como:

- data = 0.0: En este caso, el motor permaneció detenido, lo cual indica que el sistema interpreta correctamente el valor cero como ausencia de señal PWM, evitando el suministro de potencia al motor.
- data = 1.0: Al enviar este valor, el motor giró a su máxima velocidad en una dirección determinada. Esto confirma que el sistema es capaz de mapear correctamente el valor máximo positivo al mayor ciclo de trabajo permitido por la señal PWM, así como establecer la dirección de giro correspondiente.
- data = -0.5: Para este valor, el motor giró en el sentido contrario al observado con el valor positivo, a una velocidad aproximada al 50% de su capacidad máxima. Esto demuestra que el signo del valor enviado es interpretado adecuadamente como un cambio en la dirección de giro, mientras que su magnitud regula la velocidad del motor.

A partir de estas pruebas, se verificó que el sistema responde correctamente a los distintos valores enviados desde ROS 2, logrando regular tanto la velocidad como la dirección del motor DC mediante la señal PWM generada en el ESP32.

Adicionalmente, se incluye en la parte inferior de esta sección una liga a un video demostrativo en el cual se observa el funcionamiento del motor al recibir distintos valores de entrada dentro del intervalo [-1.0, 1.0], permitiendo apreciar visualmente la variación en la velocidad y el cambio en la dirección de giro conforme a los comandos enviados desde el sistema.

Resultados.mp4

Evidencia de conexión con el micro-ROS Agent

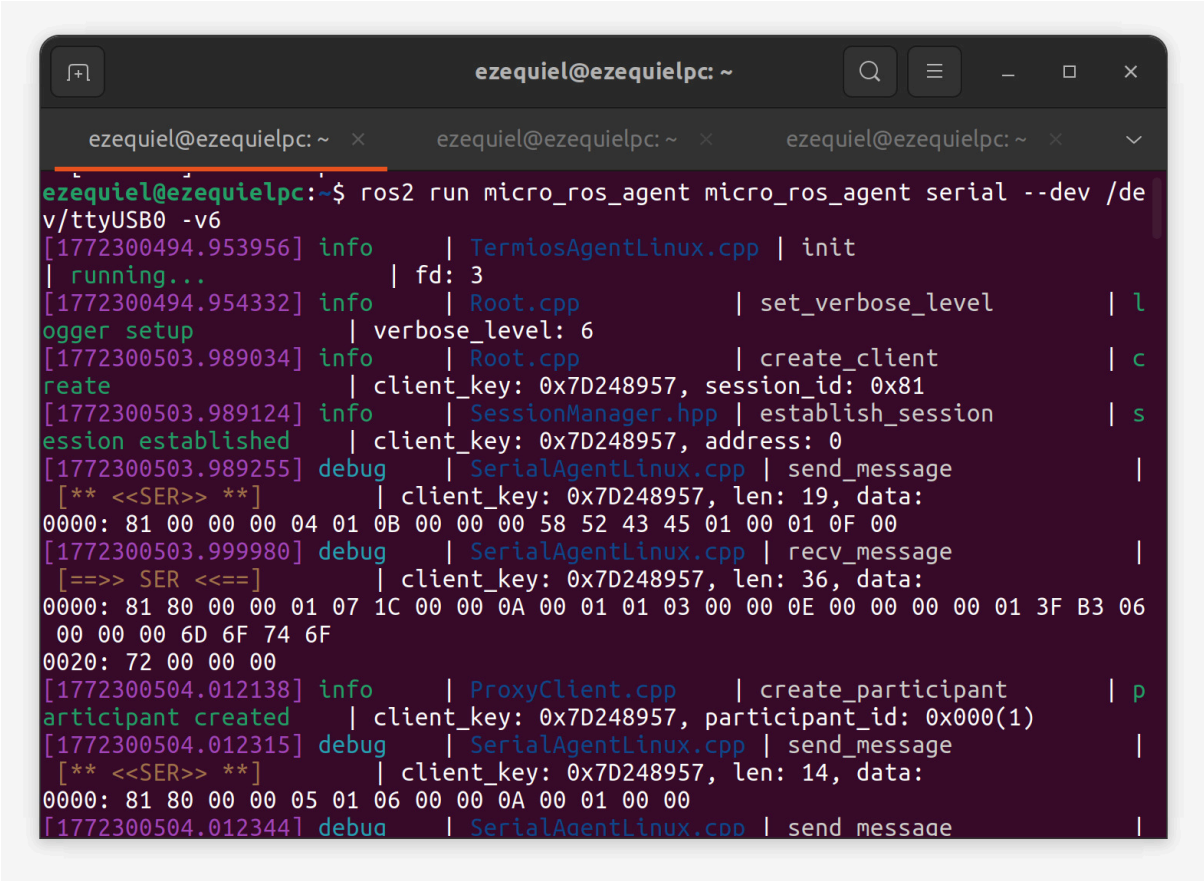

Figura 7. Inicialización del micro-ROS Agent.

En la imagen se observa la ejecución del siguiente comando:

- ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -v6

En la terminal se muestran mensajes como:

- session established
- create_client
- create_participant
- participant created

Estos mensajes indican que:

- El agente fue iniciado correctamente.
- El microcontrolador (ESP32) logró establecer una sesión con el agente.

- Se creó el cliente DDS correspondiente.
- Se registró correctamente el participante dentro del sistema ROS 2.

La aparición del mensaje "session established" confirma que la comunicación serial entre la computadora y el ESP32 fue exitosa. Asimismo, los mensajes relacionados con create_participant validan que el nodo implementado en el microcontrolador se integró correctamente al ecosistema ROS 2.

Este resultado es fundamental, ya que sin una sesión establecida entre el agente y el microcontrolador no sería posible la recepción de los mensajes enviados al tópico /cmd_pwm.

Verificación de nodos y tópicos en ROS 2
Mediante la ejecución del comando:

- ros2 node list

```
ezequiel@ezequielpc:~$ ros2 node list
/motor
```

Figura 8. Visualización de nodos activos mediante ros2 node list.

Se obtuvo como resultado la presencia del nodo /motor dentro del grafo de ROS 2. Esto confirma que el microcontrolador ESP32 ha sido correctamente reconocido como un nodo activo dentro del sistema distribuido, estableciendo comunicación con el agente micro-ROS ejecutado en la computadora.

Posteriormente, se ejecutó el comando:

- ros2 topic list

```
ezequiel@ezequielpc:~$ ros2 topic list
/cmd_pwm
/parameter_events
/rosout
```

Figura 9. Visualización de tópicos disponibles mediante ros2 topic list.

Donde se observó la existencia del tópico /cmd_pwm, el cual es utilizado para el envío de comandos de control hacia el motor. La presencia de este tópico indica que el sistema de comunicación necesario para la transmisión de datos ha sido correctamente inicializado.

Finalmente, se utilizó el comando:

- ros2 topic info /cmd_pwm

obteniendo la siguiente información:

```
ezequiel@ezequielpc:~$ ros2 topic info /cmd_pwm
Type: std_msgs/msg/Float32
Publisher count: 0
Subscription count: 1
```

Figura 10. Información del tópico /cmd_pwm mediante ros2 topic info.

El valor de Subscription count: 1 indica que existe un nodo suscrito a este tópico, el cual corresponde al nodo /motor implementado en el ESP32. Esto confirma que el microcontrolador está en espera de recibir comandos desde la computadora para regular la velocidad y dirección del motor mediante la señal PWM.

Por último durante las pruebas experimentales se observó que el motor DC no responde a valores pequeños de entrada dentro del intervalo aproximado de −0.33 a 0.31. Es decir, cuando se envían comandos dentro de ese rango normalizado, el eje del motor no genera movimiento apreciable, aunque sí puede percibirse una ligera vibración.

Este comportamiento corresponde a la llamada zona muerta (dead zone) del motor, la cual es un fenómeno común en motores DC. En nuestro caso, considerando que el motor está diseñado para operar a 6 V, el rango de activación mínima puede estimarse a partir de la normalización aplicada. Por ejemplo:

- 0.33*6V= 1.98 V
- 0.31*6V= 1.86 V

Esto indica que el motor requiere aproximadamente entre 1.8 V y 2.0 V como voltaje mínimo efectivo para vencer la fricción estática, la inercia mecánica y las pérdidas internas del

sistema. Por debajo de ese umbral, la energía suministrada no es suficiente para generar movimiento continuo, lo que explica la ausencia de giro en ese intervalo.

La vibración observada en valores cercanos al límite inferior se debe a que la señal PWM intenta energizar el motor, pero el torque generado no supera la fricción estática, produciendo pequeños intentos de movimiento sin rotación sostenida.

Este resultado es coherente con el comportamiento esperado de un motor DC real y confirma que el sistema implementado permite identificar características físicas propias del actuador, más allá del control digital programado. Como posible mejora futura, podría implementarse una compensación de zona muerta, ajustando automáticamente los valores pequeños para superar el umbral mínimo de activación y lograr una respuesta más lineal del sistema.

Conclusiones

A lo largo del desarrollo del presente reto se logró implementar un sistema distribuido de control de velocidad para un motor de corriente directa (DC) mediante la integración de ROS 2 y micro-ROS, permitiendo la comunicación entre una computadora externa y el microcontrolador ESP32. Esta comunicación hizo posible la ejecución del nodo /motor en el microcontrolador, el cual fue capaz de suscribirse al tópico /cmd_pwm y generar las señales PWM necesarias para regular tanto la velocidad como la dirección de giro del motor a través del driver puente H.

Con base en los resultados obtenidos, se puede afirmar que los objetivos planteados fueron alcanzados satisfactoriamente, ya que se logró establecer la comunicación entre los nodos del sistema, verificar la correcta suscripción al tópico de control e implementar la generación de señales PWM en el ESP32. Asimismo, se comprobó experimentalmente que el motor respondió adecuadamente a los distintos valores enviados dentro del intervalo [-1,1], permitiendo variar su velocidad y cambiar su dirección de giro conforme a los comandos publicados desde el entorno ROS 2.

Sin embargo, durante las pruebas experimentales se observó que el motor presenta vibraciones al recibir valores pequeños de PWM. Este comportamiento se debe principalmente a la existencia de una zona muerta en el motor, en la cual el voltaje promedio suministrado no es suficiente para vencer la fricción estática del sistema mecánico. Como resultado, el motor no logra generar el par necesario para iniciar un movimiento continuo, produciendo oscilaciones que se manifiestan como vibraciones.

Como posible mejora a la metodología implementada, se podría incorporar un valor mínimo de activación (offset) en la señal PWM que permita superar la zona muerta del motor, evitando así el envío de valores que no generen movimiento efectivo. De igual manera, sería posible implementar técnicas de control más avanzadas, como un controlador PID, que permitan una regulación más precisa de la velocidad del motor y una respuesta más estable ante cambios en la señal de entrada.

En conclusión, el desarrollo de este reto permitió comprender el funcionamiento de sistemas de control distribuidos en aplicaciones robóticas, así como la interacción entre ROS 2, microcontroladores y actuadores físicos, logrando cumplir con los objetivos establecidos y sentando las bases para futuras mejoras en la estabilidad y precisión del sistema de control.

Referencias Bibliográficas (APA)

- Manchester Robotics. (2024). Mini Challenge 3 (MCR2) [Presentación de PowerPoint].
- Open Robotics. (2026). ROS 2 Humble Hawksbill Documentation. Recuperado de https://docs.ros.org/en/humble/
- micro-ROS. (2026). micro-ROS documentation. https://micro.ros.org/
- Espressif Systems. (2026). ESP-IDF Programming Guide. https://docs.espressif.com/